Homework 4.4

Abhiraj Srivastava

2026-04-15

Section 1: Busy Airport

The following table tracks the annual passenger traffic for a selection of major global hubs from 2020 to 2024. Most airports show a significant recovery trend following the global travel downturn in 2020. Notably, Hartsfield-Jackson Atlanta International Airport (ATL) consistently leads the group, with its passenger count climbing from approximately 42.9 million in 2020 to over 100 million by 2023. Beijing Daxing (PKX) shows the most dramatic fluctuations, appearing in the top rankings in 2021 and 2024 but falling outside the top 50 in the intervening years.

Table 1: Total Annual Passengers 2020-2024 for Selected Global Airports

Airport Name	IATA Code	2020	2021	2022	
Hartsfield–Jackson Atlanta International Airport	ATL/KATL	42,918,685	75,704,760	93,699,630	104
Los Angeles International Airport	LAX/KLAX	28,779,527	48,007,284	65,924,298	75,
Dubai International Airport	DXB/OMDB	25,836,771	29,110,609	66,069,981	86,
Heathrow Airport	LHR/EGLL	22,111,469	—	61,614,508	79,
Frankfurt Airport	FRA/EDDF	18,768,601	24,812,849	48,918,482	59,
Beijing Daxing International Airport	PKX/ZBAD	—	25,051,012	—	

Note: Source: Wikipedia. Values represent total passenger throughput. '—' indicates the airport was outsid

Looking at Figure 1, you can see the "V-shape" recovery where travel basically bottomed out in 2020 but bounced back fast once restrictions were lifted. By the time 2024 hit, there was a huge spike in numbers, showing that people are definitely back to traveling as much as they did before the pandemic.

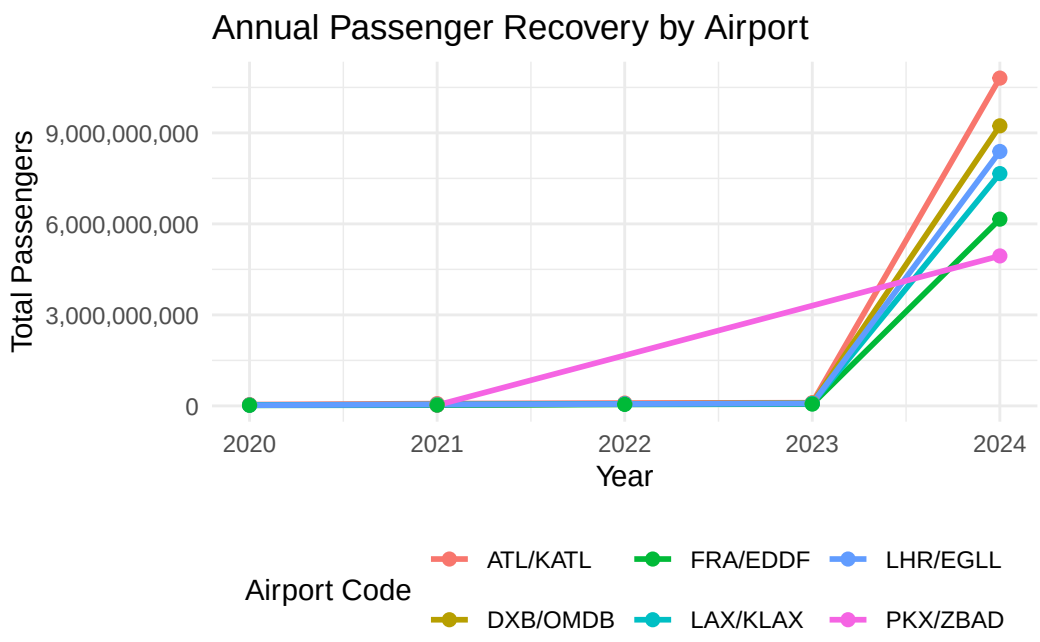


Figure 1: Passenger Traffic Trends (2020-2024)

Section 2: Monte Carlo Numerical Integration

The small multiple visualization illustrates how Monte Carlo integration accuracy improves as the sample size increases. At lower resolutions, such as $n = 10$ and $n = 100$, the area estimates vary widely (ranging from 0.8 to 0.72) because the sparse sampling fails to consistently capture the distribution's shape. By the time the simulation reaches $n = 10,000$, the estimate stabilizes at 0.9908, nearly converging to the exact mathematical answer of 1.0. This result is expected because the Chi-square distribution is a valid probability density function, meaning its total theoretical area must equal exactly 1. The small remaining deviation (the difference between 0.9908 and 1.0) is a result of two factors:The inherent randomness of the Monte Carlo method.The x bounds of $[0, 20]$ not capturing the infinitely small "tail" of the distribution that extends to the right.

Section 3 : Planning and Prompting

My Generative AI Plan

To move beyond a generic visualization, I developed a structured plan focused on data integrity and professional communication:

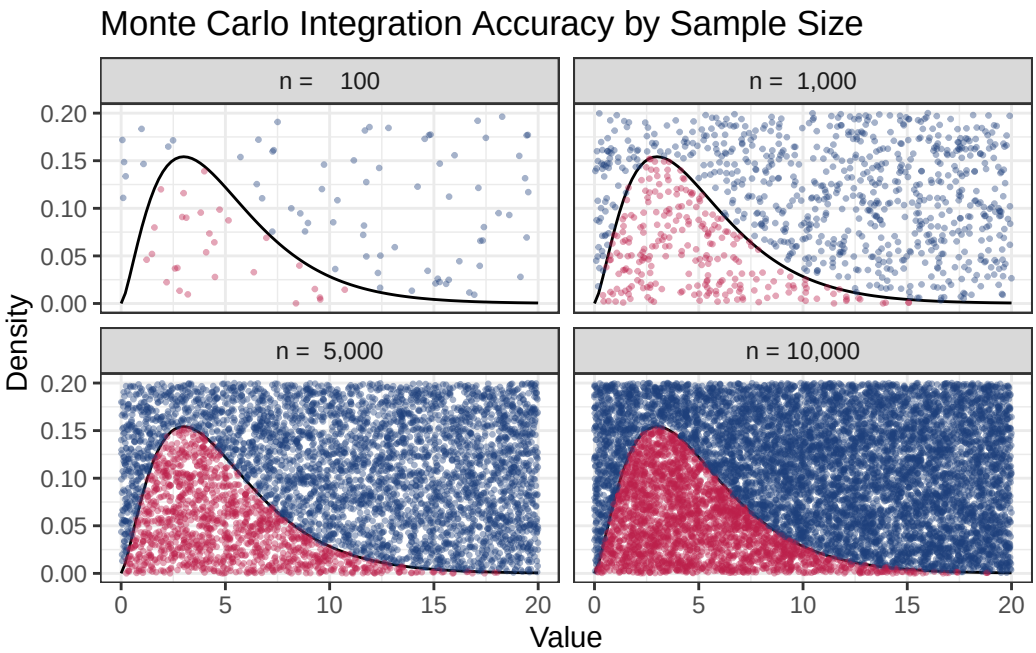


Figure 2: Monte Carlo Numerical Integration: Chi-square Distribution (df=5)

Main Goal: Compare how ulnar calcium levels change over 4 years between the null treatment group and the diet/exercise group.

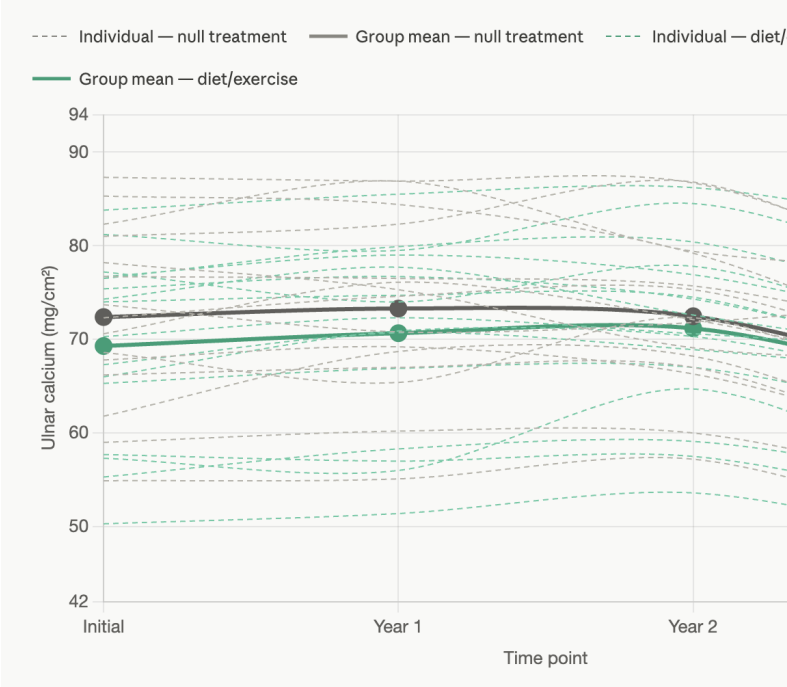
Sub-goal 1 — Read and split the raw data: Import the CSV file, select columns 1–4 (null group) and 5–8 (diet/exercise), and assign treatment labels to each.

Sub-goal 2 — Tidy the data: Use a pivot function to transform the wide table into a long table with dedicated Time and Calcium columns, then stack and verify the combined data.

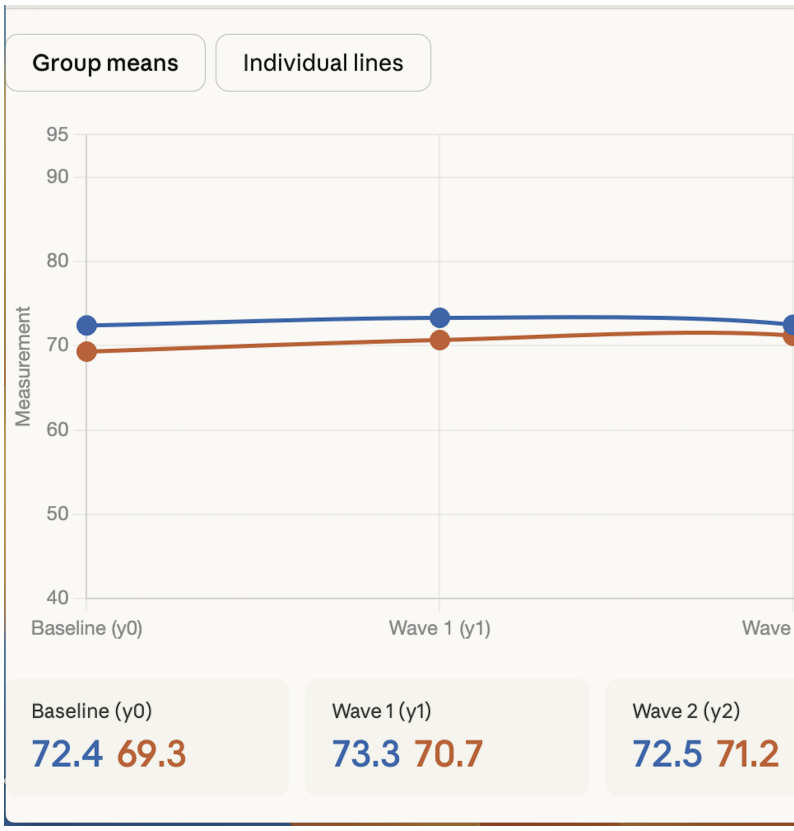
Sub-goal 3 — Plot the data: Use a tidy data frame to map the x-axis (time) and y-axis (calcium), then draw and overlay individual lines with group mean lines, distinguished by color.

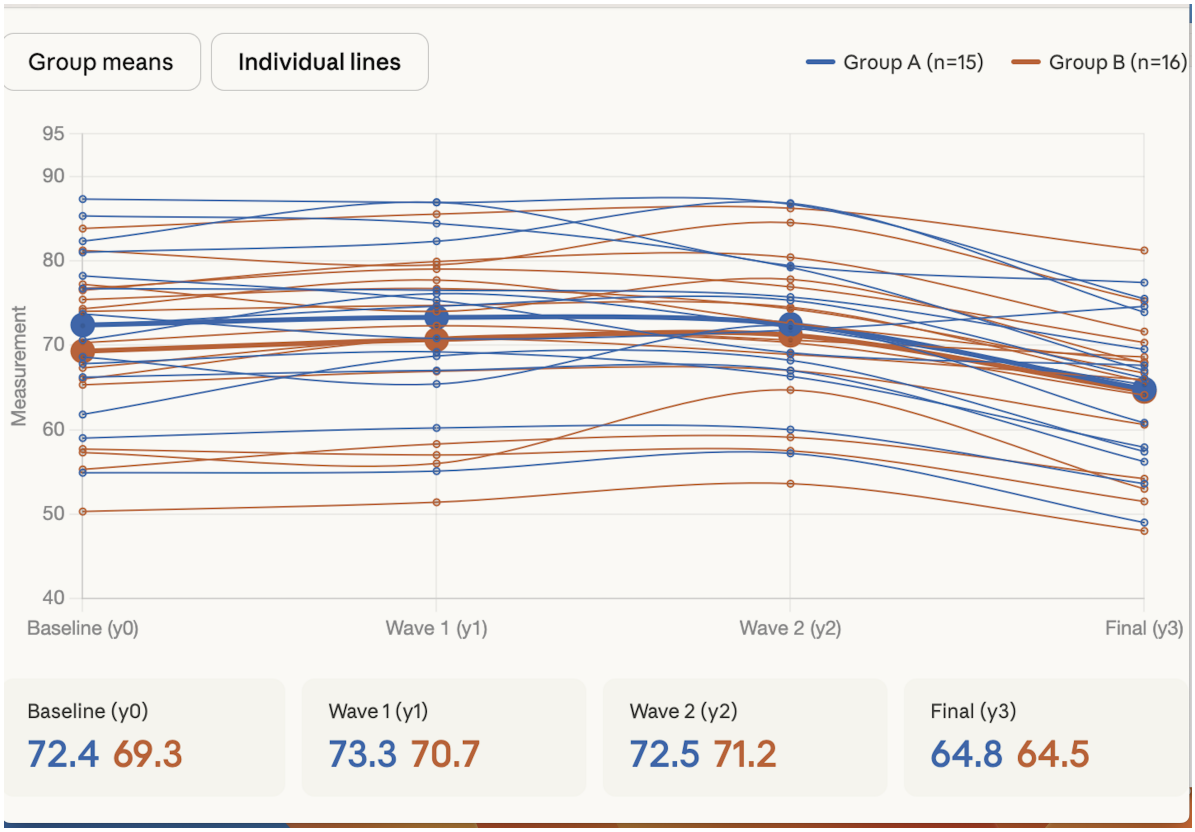
Output: A tidy data frame and a line plot showing each woman's trajectory and group means side by side.

#Response From Claude w/ Specific Prompting :



#Response From Claude w/ Genreal Prompting:





Comparison of Results

Clear Data vs. Over-simplifying: The simple plot only showed the average for each group, which hid most of the actual results. By following my plan to map every person's data, the graph shows how much people actually differed. Showing every participant's line makes the study much more honest about how much the numbers vary from person to person.

The Power of Tidy Data: The first attempt didn't organize the data correctly, so it couldn't treat time as a continuous path. By using my plan to "pivot" the data into the right format, the AI was able to treat time correctly. This made it possible to draw thick group average lines on top of the thinner individual lines, which makes it much easier to see the real difference between the treatment groups.

Section 4 : Planning and Prompting

Course Reflections

Since the start of the course, I have significantly expanded my technical toolkit for data science and reproducible reporting. Specifically, I have learned the following:

Mastering Quarto: I have moved from basic coding to understanding how to structure a full report using Quarto. This includes managing the YAML header, balancing narrative text with reproducible code, and using Markdown hierarchy to create organized documents.

Planning: I learned how to properly plan and think my thoughts out using the PCIP. This formatting and structure has helped make coding a lot easier. An example, is this assignment I had to plan first what to write and what sections go where.

Data Visualization and Tables: I have learned how to create complex visualizing tables and "small multiple" plots. For example I learned in this assignment that the function facet_wrap. Can help create a long lists of plots into a tidy gird.

GenAI Apendix:

Tool: Gemini 2.5 Flash

Date: April 15, 2026

Prompt : How to import screnshots into RStudio Answer : Add it to your R Markdown file

Use this:

Add it to your R Markdown file

Use this:

Graph Using My Specific Prompting

Tool: Claude

Date: April 15, 2026

Prompt : My Plan : [Inserted Plan] (See AI Plan Section 3) Claude : Gave me a plotted data (See Section 3)

Code Appendix

```
library(tidyverse)
library(rvest)
library(kableExtra)

# Scrape airport traffic data from Wikipedia
url <- "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"
airportWikiData <- read_html(url) |>
  html_elements(css = "table") |>
  html_table()

# Assign years to the different tables found on the page
airports_2024 <- airportWikiData[[2]] |> mutate(Year = 2024)
airports_2023 <- airportWikiData[[3]] |> mutate(Year = 2023)
airports_2022 <- airportWikiData[[4]] |> mutate(Year = 2022)
airports_2021 <- airportWikiData[[5]] |> mutate(Year = 2021)
airports_2020 <- airportWikiData[[6]] |> mutate(Year = 2020)

# Define our target list of airports
targetAirports <- c("ATL", "FRA", "PKX", "DXB", "LHR", "LAX")
```

```r
# Combine and clean the data
airport_data <- bind_rows(airports_2020, airports_2021, airports_2022, airports_2023, airport
  rename(
    Airport = matches("Airport"),
    IATA = matches("Code"),
    Passengers = matches("Total passengers|Totalpassengers")
  ) |>
  mutate(
    # Ensure we only extract the numeric part to avoid parsing errors
    Passengers = as.numeric(gsub("[^0-9]", "", as.character(Passengers)))
  ) |>
  filter(str_detect(IATA, paste(targetAirports, collapse = "|")))

# Create a wide version for the summary table
# Renaming columns to have no underscores/periods for a professional look
airport_wide <- airport_data |>
  dplyr::select(Year, Airport, IATA, Passengers) |>
  pivot_wider(
    names_from = Year,
    values_from = Passengers
  ) |>
  rename(`Airport Name` = Airport, `IATA Code` = IATA)

options(knitr.kable.NA = "-")

# Render the professional table
airport_wide |>
  kable(
    align = "llccccc",
    format.args = list(big.mark = ","),
    col.names = c("Airport Name", "IATA Code", "2020", "2021", "2022", "2023", "2024")
  ) |>
  kable_classic(full_width = F, html_font = "Cambria") |>
  row_spec(
    row = which(airport_wide$`IATA Code` == "ATL"),
    background = "#FFFFC5"
  ) |>
  footnote(
    general = "Source: Wikipedia. Values represent total passenger throughput. '-' indicates
    footnote_as_chunk = TRUE
  )
```

```r
ggplot(airport_data, aes(x = Year, y = Passengers, color = IATA, group = IATA)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_y_continuous(labels = scales::comma) +
  theme_minimal() +
  labs(
    title = "Annual Passenger Recovery by Airport",
    x = "Year",
    y = "Total Passengers",
    color = "Airport Code"
  ) +
  theme(legend.position = "bottom")

set.seed(184)
psuPalette_mc <- c('#1E407C', '#BC204B', '#3EA39E', '#E98300')
resolutions <- c(100, 1000, 5000, 10000)

sim_data <- map_df(resolutions, function(n_val) {
  data.frame(
    x = runif(n_val, min = 0, max = 20),
    y = runif(n_val, min = 0, max = 0.2),
    n = n_val
  )
}) |>
  mutate(
    f_x = dchisq(x, df = 5),
    flag = if_else(y <= f_x, 'on/below', 'above'),
    n_label = factor(paste("n =", format(n, big.mark = ",")),
                     levels = paste("n =", format(resolutions, big.mark = ",")))
  )

ggplot(sim_data, aes(x = x, y = y, color = flag)) +
  stat_function(fun = dchisq, args = list(df = 5), color = 'black', xlim = c(0, 20)) +
  geom_point(alpha = 0.4, size = 0.5) +
  scale_color_manual(values = psuPalette_mc) +
  facet_wrap(~ n_label, ncol = 2) +
  theme_bw() +
  theme(legend.position = 'none') +
  labs(
    title = "Monte Carlo Integration Accuracy by Sample Size",
    x = "Value",
    y = "Density"
```

```
)
```